

Handling Aperiodic Overloads — Buttazzo

Milestone 2: Contextualization and Tradeoff Analysis

Rionela Kovaci

Hamm-Lippstadt University of Applied Sciences

Real-Time Systems Seminar

rionela.kovaci@stud.hshl.de

I. NEIGHBORING APPROACHES

There are several ways to deal with aperiodic tasks in real-time systems. Each approach works differently and has its own problems.

- **Plain EDF** — runs all tasks by earliest deadline. Does not check if the system is overloaded. When too many tasks arrive, all tasks can miss their deadlines [3].
- **GED** — checks if a new task fits before accepting it. If not, the new task is rejected. Does not consider how important a task is [1].
- **RED** — checks if a new task fits. If not, it removes the least important task instead. Also tries to reuse saved CPU time. Protects the most critical tasks even during overload [1].
- **TBS / Sporadic Server** — gives each aperiodic task a limited CPU budget. Overload is avoided because tasks cannot use more than their budget [2].
- **Rate Monotonic** — assigns fixed priorities before the system runs. Tasks with shorter periods get higher priority. Cannot react to unexpected overload [3].

II. COMPARISON TABLE

TABLE I
COMPARISON OF APPROACHES

Approach	Online	Value-based	Reclaiming	Critical class
Plain EDF	yes	no	no	no
GED	yes	no	no	no
RED	yes	yes	yes	yes
TBS/SS	yes	no	no	no
RM	no	no	no	no

III. KEY TRADEOFFS

- **Online vs. offline** — RED decides at runtime because task arrivals are not known in advance. Offline methods like RM cannot handle this.
- **Best result vs. fast decision** — finding the best set of tasks to reject is too slow. RED uses a simple rule: reject the least important task. This is fast but not always perfect.
- **Worst case vs. real behavior** — RED uses worst-case execution times, which are often too high. Tasks usually

finish earlier. Resource reclaiming uses this saved time so fewer tasks are rejected.

- **Deadline tolerance** — allowing a task to finish a little late reduces unnecessary rejections. But if the tolerance is too large, the result may arrive too late to be useful.
- **Task value** — assigning a value to each task gives more control over what gets protected. But someone has to decide these values, which is not easy in practice.

IV. EMBEDDED SYSTEMS CONSEQUENCES

- Most embedded operating systems like FreeRTOS and AUTOSAR use fixed priorities, not EDF. RED cannot be used directly on these systems.
- RED only handles aperiodic tasks. In real systems, periodic and aperiodic tasks run together. This combination is not covered by RED.
- RED needs four queues running at the same time. This uses memory. On small microcontrollers this can be a problem.
- RED needs to know the worst-case execution time of each task when it arrives. This is not always easy to measure in real systems.
- RED only works on one processor. Many modern embedded systems have multiple cores. RED has no solution for this.

REFERENCES

- [1] G. C. Buttazzo and J. A. Stankovic, “Adding Robustness in Dynamic Preemptive Scheduling,” Kluwer Academic Publishers, 1995.
- [2] M. Spuri and G. C. Buttazzo, “Scheduling Aperiodic Tasks in Dynamic Priority Systems,” *Real-Time Systems*, vol. 10, pp. 179–210, 1996.
- [3] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.